



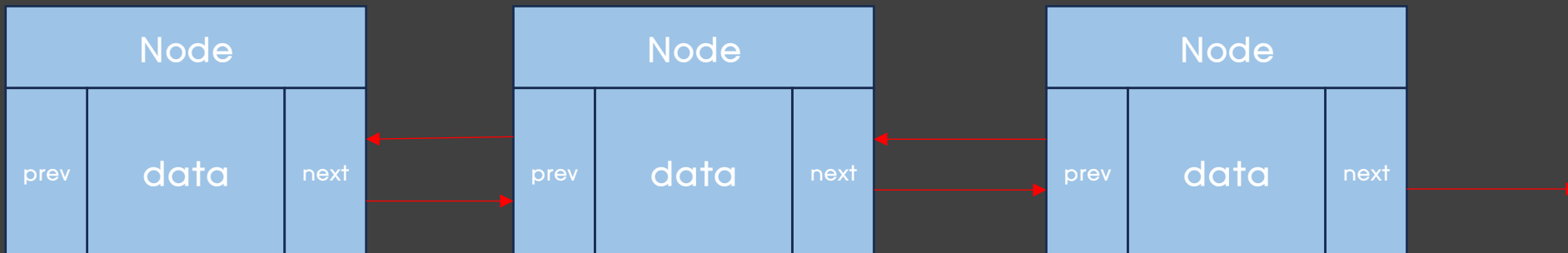
과제 가이드

컴퓨터학부 박한영



연결 리스트

- 연결 리스트는 데이터를 저장하는 노드들이 포인터로 연결된 자료구조
- 배열과 다르게 연속된 메모리에 저장된 것이 아님
- 배열과 다르게 중간 삽입/삭제가 비교적 쉽다.
- 배열과 다르게 인덱스로 접근이 어렵고, 순차적으로 탐색해야 한다.



과제 안내

- 링크로 들어가 과제 수락
- 또는 리포지토리를 포크하여 작업하여도 됨
- git 사용이 어려운 경우, 리포지토리에서 파일을 다운받은 후 작성하여 멘토에게 제출
- 리포지토리 링크: https://github.com/JaramStudy/C_Study_LinkedList



과제 안내

- 주어진 `linked_list` 파일들을 작성하여 연결리스트 자료구조를 구현한다.
- `index_at`: `Node node = *head;` 후 반복문을 사용하여 `index`번째의 노드를 반환
힌트: `for(int i = 0; i < index; i++) node = node.next;` 와 같이 자신을 참조
- `end`: `IndexAt`과 비슷하나 리스트의 마지막 노드까지 탐색하여 마지막 노드를 반환
힌트: 위와 같이 반복문으로 `node = node.next`처럼 참조하면서 `next`가 `NULL`일때까지 탐색
- `size`: 리스트의 크기를 반환
힌트: 리스트를 탐색하면서 노드의 개수를 세도 되고, 메인 함수에 노드의 개수를 기록하는 변수를 만들고 이 함수를 없애도 됨
- `insert_at`: `index`번째에 `data`를 가진 노드를 생성해 삽입
힌트: `index_at`과 비슷하게 탐색하고, 기준점이 되는 노드 앞 또는 뒤에 새 노드를 삽입할 때, 새 노드의 `prev`와 `next`를 먼저 설정하고, 앞/뒤 노드의 `next/prev`를 새 노드로 설정
- `insert`: 가장 마지막 노드 뒤에 `data`를 가진 노드를 생성해 삽입
힌트: 위와 같이 새 노드의 `prev`를 먼저 설정하고 원래 노드의 `next`를 새 노드로 설정, 새 노드의 `next`는 `NULL`로 설정
- `delete`: `index`번째의 노드를 삭제
힌트: 삭제할 노드의 앞 노드의 `next` = 삭제할 노드의 뒤 노드, 삭제할 노드의 뒤 노드의 `prev` = 삭제할 노드의 앞 노드로 설정한 후 삭제하려는 노드를 삭제
- `print`: 리스트의 모든 원소를 출력(리스트를 순차적으로 탐색)

